

252-0027

**Einführung in die Programmierung
Übungen**

Woche 14: Exam Tipps

**Joran Bühler
Departement Informatik
ETH Zürich**

Collections

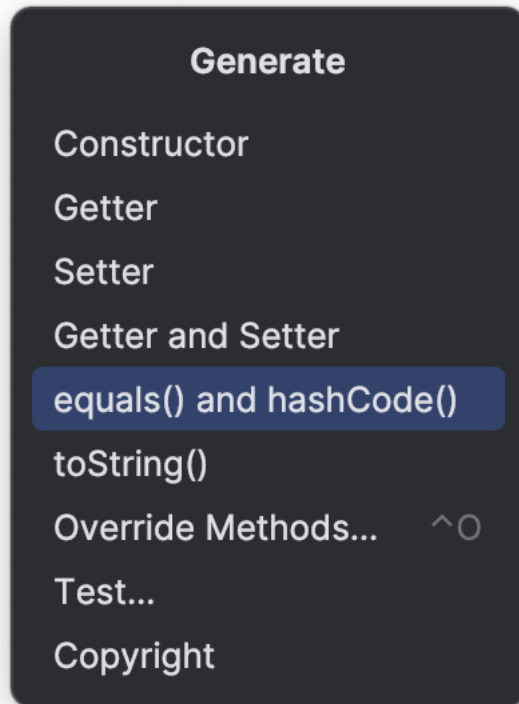
- **HashSet – nicht definiert.**
- **LinkedHashSet – wie eingefügt.**
- **ArrayList – wie im Array gespeichert, entsprechend dem Index**
- **LinkedList – wie in der Liste gespeichert.**
- **TreeSet – aufsteigend nach natürlicher Ordnung (definiert durch compareTo).**
- **HashMap – nicht definiert.**
- **TreeMap – aufsteigend nach natürlicher Ordnung (definiert durch compareTo).**
- **LinkedHashMap – wie eingefügt.**

Sortiert vs Unsortiert

- `HashSet<Room>`
 - Die Methoden `equals` und `hashCode` der Objekt-Klasse überschreiben.
 - Gilt auch für `HashMap<>`
 - -> Unsortiert
- `TreeSet<Room>`
 - `Comparable-Interface` implementieren und `compareTo`-Methode überschreiben.
 - Gilt auch für `TreeMap<>`
 - -> Sortiert

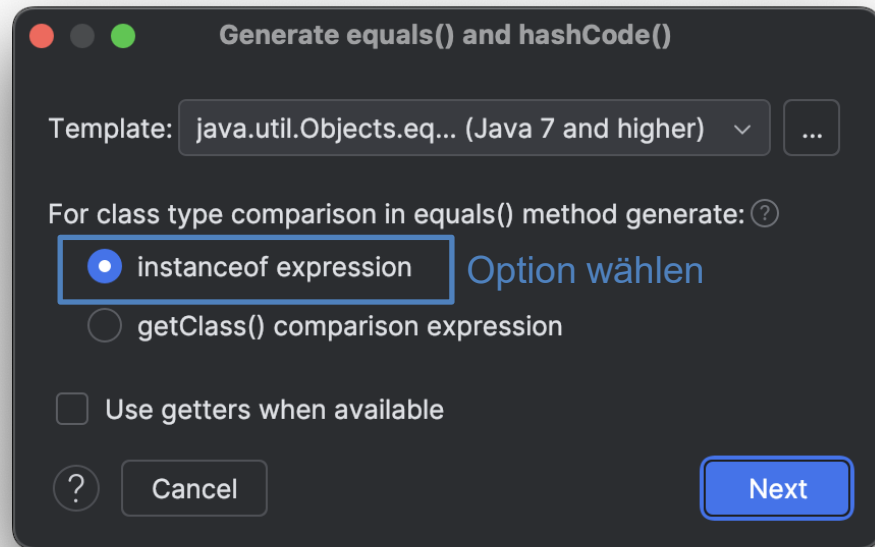
HashSet: Methoden generieren

- HashSet<Room>
 - **In IntelliJ:** Rechtsclick -> Generate... -> Generate hashCode() und equals()

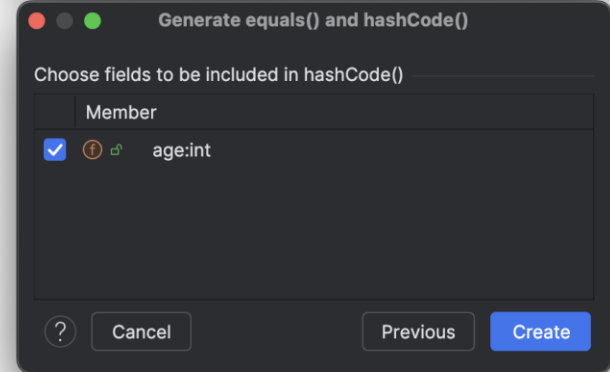
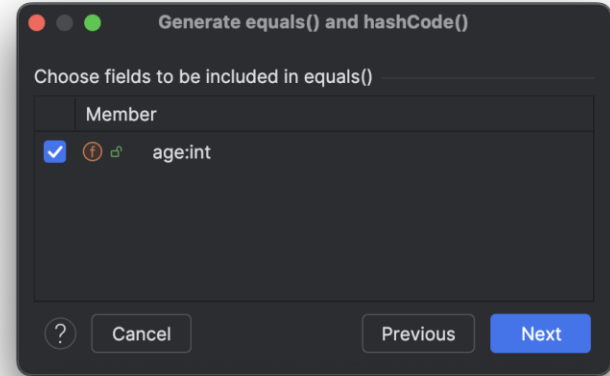


HashSet: Methoden generieren

- `HashSet<Room>`
 - **In IntelliJ:** Rechtsclick -> Generate... -> Generate hashCode() und equals()
- `getClass()` erfordert dieselbe Klasse vom anderen Objekt



- `HashSet<Room>`
 - **In IntelliJ:** Rechtsclick -> Generate... -> Generate hashCode() und equals()
 - Auswahl, welche Attribute verglichen werden sollen



HashSet: Methoden generieren

Wir benutzen Operationen und Methoden von Superklassen um hashCode zu implementieren.

```
@Override
public boolean equals(Object o) { //bad equals method. Rooms with same age are not the same!
    if (o == null || getClass() != o.getClass()) return false;
    Room room = (Room) o;
    return age == room.age;
}

@Override
public int hashCode() { //bad hashCode method. Rooms with same age are not the same!
    return Objects.hashCode(age);
}
```

Comparable

```
public interface Comparable<T>
```

This interface imposes a total ordering on the objects of each class that implements it. This ordering is referred to as the class's *natural ordering*, and the class's `compareTo` method is referred to as its *natural comparison method*.

Lists (and arrays) of objects that implement this interface can be sorted automatically by `Collections.sort` (and `Arrays.sort`). Objects that implement this interface can be used as keys in a sorted map or as elements in a sorted set, without the need to specify a comparator.

Vorteile von Comparable:

- Wenn Comparable implementiert ist, dann können wir mit `Collections.sort` sortieren.
- Wir können mittels `PriorityQueue<>` einen Heap erstellen.
- Mit `Collections.reverseOrder()` können wir von aufsteigender Ordnung zu absteigender Ordnung wechseln.

TreeSet: Comparable Implementieren

Construction of sorted collection with non-comparable elements

© java.util.TreeSet<E>

@Contract(pure = true) ↗
public TreeSet()

Constructs a new, empty tree set, sorted according to the natural ordering of its elements. All elements inserted into the set must implement the `Comparable` interface. Furthermore, all such elements must be *mutually comparable*: `e1.compareTo(e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the set. If the user attempts to add an element to the set that violates this constraint (for example, the user attempts to add a string element to a set whose elements are integers), the `add` call will throw a `ClassCastException`.

< 21 >

TreeSet: Comparable Implementieren

- TreeSet<Room>

Damit TreeSet funktioniert muss Room das Comparable-Interface implementieren.

```
public class Room implements Comparable<Room> { 2 usages
    int age; no usages

    @Override
    public int compareTo(Room o) {
        return 0;
    }
}
```

TreeSet: Comparable Implementieren

```
public class Room implements Comparable<Room> { 2 usages
    int age; no usages

    @Override
    public int compareTo(Room o) {
        return 0;
    }
}
```

Jetzt müssen wir nur noch die `compareTo`-Methode sinnvoll implementieren.

TreeSet: Comparable Implementieren

- `compareTo` gibt 0 zurück, falls die Objekte `equal` sind.
- `compareTo` gibt `>0` zurück, falls `this`-Objekt „grösser“ als das Parameter-Objekt ist.
- `compareTo` gibt `<0` zurück, falls `this`-Objekt „kleiner“ als das Parameter-Objekt ist.

TreeSet: Comparable Implementieren

```
@Override  
public int compareTo(Room o) {  
    return this.age - o.age; //simple trick to sort by Int attribute  
}
```

Beispiel mit Room

```
public class Room implements Comparable<Room>{
    public int age; 4 usages

    @Override
    public int compareTo(Room o) {
        return this.age - o.age; //simple trick to sort by Int attribute
    }

    @Override
    public String toString() {
        return "Room with Age" + age;
    }

    //Constructor:
    public Room(int age){ 4 usages
        this.age = age;
    }
}
```

Beispiel mit Room

```
public static void main(String[] args) {  
    LinkedList<Room> Rooms = new LinkedList<>();  
    Rooms.add(new Room(2));  
    Rooms.add(new Room(5));  
    Rooms.add(new Room(3));  
    Rooms.add(new Room(1));  
  
    //toString of LinkedList calls toString of Room  
    System.out.println(Rooms); //[Room with Age2, Room with Age5, Room with Age3, Room with Age1]  
    Collections.sort(Rooms);  
    System.out.println(Rooms); //[Room with Age1, Room with Age2, Room with Age3, Room with Age5]  
    Collections.sort(Rooms, Comparator.reverseOrder());  
    System.out.println(Rooms); //[Room with Age5, Room with Age3, Room with Age2, Room with Age1]  
}
```


Programmierprüfung importieren

1. Go to Vis Exam Collection
2. Download IntelliJ Exam Template
3. Extract and put into your Exams folder
4. File -> Open -> Choose Exam folder
5. Solve

Lernphase-Tipps by Joran:

- **Unterschätzt die Lernphase nicht**
 - Ihr könnt sehr viel nachholen
 - Es erfordert viel Disziplin
 - Schreibt einen Lernplan
- **Schlaf und Abwechslung sind wichtig!**
- **Social Media Konsum kontrollieren**
- **Balance zwischen alten Prüfungen und Theorie**
- **Eventuell an der ETH mit Freunden lernen**
- **Für die, die knapp sind: Wie viel Diskmath lohnt sich?**

Vor den Prüfungen

- **Schlaf ist wichtig -> frischer Kopf**
 - In der Nacht davor zu lernen bringt nichts
- **Nehmt Snacks und Getränke mit**
- **Nehmt Legi und Stifte mit**
 - Leuchtstifte für Eprog!
- **Zusammenfassung ausdrucken**
- **Zum richtigen Raum gehen. -> evtl in Oerlikon!**

Programmierprüfung

- **Löst Altprüfungen**
- **Tricks in den Lösungen von anderen finden**
 - Benutzt public Attribute
 - This Objekt in Konstruktor übergeben
 - Human soll Wissen in welchem Game er spielt
 - Collections verwenden (Collections.sort())
 - Bei Strukturierten Graphen Aufgaben evtl Nodes in Matrix speichern
 - Shortcuts und Unterstützung von IntelliJ benutzen
- **Lest alles durch und markiert mit Leuchtstiften**
 - Grober Plan, wie man es löst. -> Evtl skizzieren?
 - Ist Teilaufgabe b) einfacher?
- **Lieber etwas länger planen als sinnlos debuggen**
- **Lösung hinterfragen**
 - Edge cases?

Theorieprüfung

- Übung macht den Meister
- Immer sehr ähnlich -> Alte Prüfungen lösen
- Welche Aufgaben dauern zu lange?
 - Generell hat man in Eprog sehr wenig Zeit
- Bei Vererbungsaufgaben Bäume zeichnen
- In EBNF Muster erkennen

Gohars Website



<https://eprog23.wixstudio.com/hs23/lernphase>

Frohe Weihnachten

Danke für alles

Hat Spass gemacht mit euch!